HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

# THỰC TẬP CƠ SỞ

## Weekly Report

## Đề tài: XÂY DỰNG HỆ THỐNG GỢI Ý SẢN PHẨM

## Giai đoạn 4: Đánh giá và tối ưu

| | |
|---|---|
| **Giảng viên** | **: Kim Ngọc Bách** |
| **Họ và tên sinh viên** | **: Nguyễn Văn Trường** |
| **Mã sinh viên** | **: B23DCCN870** |
| **Lớp** | **: D23CQCN02-B** |
| **Nhóm** | **: 11** |

***Hà Nội – 2026***

# Nội dung

I. Huấn luyện mô hình ..... 3
1. GRID SEARCH ..... 3
II. Huấn luyện lại với tham số tốt nhất ..... 4
III. Gợi ý Top-10 cho demo-user ..... 5

# I. Huấn luyện mô hình

## 1. GRID SEARCH

Grid search (tìm kiếm lưới) là một kỹ thuật tối ưu hóa tham số (hyperparameter tuning) phổ biến trong Machine Learning. Mục tiêu của nó là tìm ra bộ thông số tốt nhất cho một mô hình cụ thể để đạt được hiệu suất cao nhất.

Grid search thực hiện một cách tiếp cận "vét cạn" (brute-force). Bạn cung cấp một danh sách các giá trị cho các tham số khác nhau, và thuật toán sẽ:

1. Tạo ra tất cả các tổ hợp (combinations) có thể có từ các giá trị đó.
2. Huấn luyện mô hình với từng tổ hợp.
3. Đánh giá hiệu suất (thường thông qua **Cross-Validation**).
4. Chọn ra tổ hợp có kết quả tốt nhất.

```
from itertools import product as iproduct

K_grid      = [10, 20, 50]
alpha_grid  = [0.005, 0.01]
lambda_grid = [0.01, 0.02]

# Dùng 30% dữ liệu để grid search
df_gs_train = df_train.sample(frac=0.3, random_state=42)
df_gs_val   = df_test.sample(n=2000, random_state=42)

gs_results = []
print(f"  Grid search: {len(K_grid)*len(alpha_grid)*len(lambda_grid)} tổ hợp\n")
print(f"  {'K':>4} {'alpha':>7} {'lambda':>8} {'Val RMSE':>10}")
print(SEP2)

for K, alpha, lam in iproduct(K_grid, alpha_grid, lambda_grid):
    m = MatrixFactorization(K=K, alpha=alpha, lambda_=lam, n_epochs=10, seed=42)
    m.fit(df_gs_train, len(uid2idx), len(mid2idx), uid2idx, mid2idx,
          df_val=df_gs_val, verbose=False)
    val_rmse = m.val_loss_history[-1]
    gs_results.append((K, alpha, lam, val_rmse))
    print(f"  {K:>4} {alpha:>7.4f} {lam:>8.3f} {val_rmse:>10.4f}")

best = min(gs_results, key=lambda x: x[3])
print(f"\n  ➡  Tốt nhất: K={best[0]}, α={best[1]}, λ={best[2]}  →  RMSE={best[3]:.4f}")
```

```
Grid search: 12 tổ hợp

    K   alpha   lambda   Val RMSE
------------------------------------------------------------
   10  0.0050    0.010     1.0422
   10  0.0050    0.020     1.0424
   10  0.0100    0.010     1.0239
   10  0.0100    0.020     1.0240
   20  0.0050    0.010     1.0418
   20  0.0050    0.020     1.0420
   20  0.0100    0.010     1.0242
   20  0.0100    0.020     1.0242
   50  0.0050    0.010     1.0452
   50  0.0050    0.020     1.0452
   50  0.0100    0.010     1.0286
   50  0.0100    0.020     1.0281

 ➡  Tốt nhất: K=10, α=0.01, λ=0.01  →  RMSE=1.0239
```

## II. Huấn luyện lại với tham số tốt nhất

```python
# ── HUẤN LUYỆN LẠI VỚI THAM SỐ TỐT NHẤT ──────────────────

best_K, best_alpha, best_lambda = best[0], best[1], best[2]

mf_best = MatrixFactorization(
    K        = best_K,
    alpha    = best_alpha,
    lambda_  = best_lambda,
    n_epochs = 50,
    seed     = 42
)
mf_best.fit(df_train, len(uid2idx), len(mid2idx),
            uid2idx, mid2idx, df_val=df_test, verbose=True)

# Đánh giá lần cuối
final_preds, final_actuals = [], []
for _, row in df_test.iterrows():
    pred = mf_best.predict(int(row["userId"]), int(row["movieId"]))
    final_preds.append(pred)
    final_actuals.append(row["rating"])

mf_best_mae  = mean_absolute_error(final_actuals, final_preds)
mf_best_rmse = np.sqrt(mean_squared_error(final_actuals, final_preds))
print(f"\n  MAE (best MF): {mf_best_mae:.4f}")
print(f"  RMSE (best MF): {mf_best_rmse:.4f}")
```

```
Bắt đầu huấn luyện: K=10, α=0.01, λ=0.01, epochs=50
 Epoch   Train RMSE   Val RMSE
------------------------------------------------------------
     1        1.0063      1.0557
     5        0.9181      1.0005
    10        0.8808      0.9860
    15        0.8234      0.9792
    20        0.7809      0.9840
    25        0.7534      0.9925
    30        0.7349      1.0011
    35        0.7218      1.0087
    40        0.7121      1.0157
    45        0.7045      1.0207
    50        0.6988      1.0271

Huấn luyện xong!  Train RMSE cuối = 0.6988

MAE (best MF): 0.8011
RMSE (best MF): 1.0317
Chỉ số                  Giá trị
-------------------------------------------
MAE                      0.7739
RMSE                     0.9809
```

## III. Gợi ý Top-10 cho demo-user

```python
DEMO_USER = 1
mf_recs = mf_best.recommend(DEMO_USER, df_train, movie_title, N=10)

print(f"  Top-10 gợi ý cho User {DEMO_USER} (Matrix Factorization):\n")
print(f"  {'#':<4} {'Tên phim':<50} {'Pred Rating':>11}")
print(SEP2)
for rank, (mid, score) in enumerate(mf_recs, 1):
    title = movie_title.get(mid, f"Movie {mid}")[:48]
    print(f"  {rank:<4} {title:<50} {score:>11.3f}")
```

Kết quả đạt được:

```
Top-10 gợi ý cho User 1 (Matrix Factorization):

#    Tên phim                                          Pred Rating
------------------------------------------------------------
1    Wrong Trousers, The (1993)                              5.000
2    Leaving Las Vegas (1995)                                5.000
3    Boogie Nights (1997)                                    5.000
4    Close Shave, A (1995)                                   5.000
5    Duck Soup (1933)                                        5.000
6    Dr. Strangelove or: How I Learned to Stop Worryi        5.000
7    Paths of Glory (1957)                                   5.000
8    Ran (1985)                                              5.000
9    When We Were Kings (1996)                               5.000
10   Top Hat (1935)                                          5.000
```